

CTR Modeling for Contextual Advertising in LLM-based Systems: The Adstract Framework

Adstract

September 2025

Abstract

Click-Through Rate (CTR) estimation is the central component in contextual advertising platforms. It governs auction fairness, publisher payouts, advertiser billing, and system-wide optimization. This document presents a practical framework for CTR modeling in the Adstract platform, starting from Bayesian priors at cold start, incorporating posterior updates with impressions and clicks, and extending toward usage in auctions, dashboards, and machine learning. The methodology is designed for stability, interpretability, and extensibility, ensuring that CTR estimates can be trusted in production from the very first ad request.

1 Introduction

Click-Through Rate (CTR) modeling is a foundational problem in digital advertising. Large platforms such as Google, Meta, and major DSPs all rely on CTR estimation pipelines combining Bayesian smoothing, hierarchical shrinkage, and machine learning. These methods stabilize estimates at low data volumes, provide reliable priors for auctions, and enable fair reporting to advertisers and publishers.

In the Adstract platform, the same principles apply, but with important contextual differences. Unlike traditional web advertising where static banners or sponsored search results may accumulate millions of impressions per creative, ads in our system are injected dynamically into chatbot responses. This introduces three unique challenges:

1. **Higher variance.** CTR depends strongly on the surrounding conversation text and how the LLM weaves the ad into the response, leading to larger fluctuations than in static display environments.
2. **Smaller impression volumes per unit.** Each conversation prompt is unique, and the same ad may only appear in hundreds or thousands of opportunities, not millions. This makes smoothing and priors more critical.

3. **Contextual dependencies.** CTR is influenced not only by the ad itself, but also by the publisher domain (horizontal aggregator vs. vertical chatbot) and the semantic category of the conversation. This introduces strong cross-feature effects absent in simpler banner placements.

As a result, while Adstract builds on the same Bayesian and statistical foundations used across the ad industry, it must place greater emphasis on priors, variance-based guardrails, and contextual modeling. Reliable CTR estimates are essential both for accurate auction pricing (e.g. unifying CPC and CPM bids into effective CPMs) and for sustaining trust across advertisers, publishers, and end-users.

This document outlines our full CTR modeling framework, starting from Bayesian priors at cold start, through posterior updates and guarded weight learning, and extending to applications in auctions, dashboards, and machine learning pipelines.

2 CTR Modeling and Application

CTR modeling in Adstract relies on a Bayesian smoothing framework with logit-based priors. At each ad opportunity, the system must decide whether to serve a candidate ad, estimate its expected CTR, and translate this into effective CPM units for auction ranking. This section describes the full process, from prior definition to posterior computation, auction integration, dashboard reporting, and ML readiness.

2.1 Bayesian Prior at Cold Start

CTR is modeled as a random variable with a Beta prior distribution:

$$CTR \sim \text{Beta}(\alpha_0, \beta_0).$$

The prior mean m_0 is computed in logit space:

$$m_0 = \sigma(z_0), \quad z_0 = w_0 + w_{\text{ad}}[a] + w_{\text{dom}}[d] + w_{\text{ctx}}[c],$$

where w_0 is the global intercept, and $w_{\text{ad}}, w_{\text{dom}}, w_{\text{ctx}}$ are feature weights for the ad category, publisher domain type, and conversation category respectively.

The prior parameters are defined as:

$$\alpha_0 = m_0 \cdot s, \quad \beta_0 = (1 - m_0) \cdot s,$$

where s is the prior strength (pseudo-impressions). This ensures stability at low data volumes.

2.2 Posterior Update with Observations

After I impressions and C clicks in a bucket (a, d, c) , the posterior is:

$$CTR \sim \text{Beta}(\alpha_0 + C, \beta_0 + I - C).$$

The posterior mean estimate is:

$$\widehat{CTR} = \frac{\alpha_0 + C}{\alpha_0 + \beta_0 + I}.$$

Variance is:

$$\text{Var}(CTR) = \frac{(\alpha_0 + C)(\beta_0 + I - C)}{(\alpha_0 + \beta_0 + I)^2(\alpha_0 + \beta_0 + I + 1)}.$$

Credible intervals can be derived either by quantiles of the Beta distribution or approximated as:

$$CI_{90\%} \approx \widehat{CTR} \pm 1.645 \cdot \sqrt{\text{Var}(CTR)}.$$

2.3 Application in Auctions

CTR estimates are used to unify CPC and CPM bids in eCPM space:

$$eCPM_{\text{CPC}} = CPC_{bid} \cdot \widehat{CTR} \cdot 1000,$$

$$eCPM_{\text{CPM}} = CPM_{bid}.$$

The winning ad is selected by highest eCPM. Floors and guardrails (e.g., using lower CI bounds for unstable CTRs) ensure conservative and fair outcomes.

2.4 Guard Conditions for CTR Usage

After computing the posterior CTR and its variance, we apply two guard conditions to determine whether an ad's CTR estimate is *trusted*. These guards influence both how CTR is displayed in dashboards and how it is applied in auctions.

Step 1: Variance Guard. The posterior variance of CTR is:

$$\text{Var}(CTR) = \frac{(\alpha_0 + C)(\beta_0 + I - C)}{(\alpha_0 + \beta_0 + I)^2(\alpha_0 + \beta_0 + I + 1)}.$$

We compare this to a maximum variance threshold:

$$\text{Var}_{\max}(CTR) = \left(\frac{CI_{\text{length}, \max}}{2 \cdot 1.645} \right)^2,$$

where $CI_{\text{length}, \max}$ is the maximum acceptable confidence interval width (e.g. 2 percentage points). If $\text{Var}(CTR) > \text{Var}_{\max}(CTR)$, the CTR estimate is *not trusted*. In this case:

- Dashboards do not display CTR values as “stable” (optionally marked as “Learning”).
- In auctions, the effective CTR used is the **lower bound of the confidence interval** (5th percentile of the Beta posterior).

Step 2: Stability Guard. If the variance guard is passed, we then check stability based on impressions and CI width:

$$I \geq I_{\min} \quad \text{and} \quad CI_{\text{length}} \leq CI_{\text{length},\max}.$$

If both conditions are satisfied, the ad is marked as *Trusted*:

- The CTR value is displayed in dashboards.
- The *mean posterior CTR* is used in auctions.
- The ad’s CTR can be used in downstream ML training.

If either condition fails, the ad remains *not trusted*, and again the lower CI bound is used in auctions instead of the mean.

Summary. Thus, the guard process is:

$$\text{CTR trusted} \iff \left(\text{Var}(CTR) \leq \text{Var}_{\max} \right) \wedge \left(I \geq I_{\min} \wedge CI_{\text{length}} \leq CI_{\text{length},\max} \right).$$

Otherwise, dashboards mark the CTR as unstable, and auctions use the conservative lower bound estimate.

2.5 Worked Example

Suppose baseline CTR = 1% ($w_0 = -4.595$), and we have an ad category *health*, domain type *vertical*, context *how-to*.

Step 1: Prior. Prior strength $s = 100$, $m_0 = 0.01 \Rightarrow \alpha_0 = 1, \beta_0 = 99$.

Step 2: Data. After $I = 220$ impressions, $C = 3$ clicks:

$$\alpha' = 1 + 3 = 4, \quad \beta' = 99 + (220 - 3) = 316.$$

$$\widehat{CTR} = \frac{4}{320} = 1.25\%.$$

Step 3: Auction. For a CPC bid of \$1.20:

$$eCPM = 1.20 \times 0.0125 \times 1000 = \$15.00.$$

For a competing CPM bid of \$14.50, the CPC ad wins.

Step 4: Dashboard. Compute variance and CI. If conditions satisfied, display 1.25% CTR with bounds. Otherwise mark as “Learning”.

Step 5: ML readiness. If CI passes conditions, include (a, d, c) as a stable sample in model training, weighted by impressions.

Disclaimer on Percentage Points

Throughout this document, thresholds for CTR stability and confidence intervals are expressed in *percentage points* (pp). It is important to distinguish this from a *percent change*:

- A **percentage point (pp)** refers to the *absolute difference* between two percentages. For example, moving from a CTR of 1.0% to 3.0% is an increase of **2 percentage points**.
- A **percent change**, by contrast, is the relative difference. The same increase from 1.0% to 3.0% is a 200% increase in relative terms.

In our guard conditions, when we state that the maximum allowed confidence interval width is, for example, **2 percentage points**, we mean that the upper bound minus the lower bound of the CTR confidence interval must not exceed 0.02 in absolute probability terms (i.e., 2% absolute CTR difference), not a relative percent change.

3 Weight Update Procedure (Ad, Domain, Context)

This section specifies how we *learn and maintain* the feature weights $w_{\text{ad}}[\cdot]$, $w_{\text{dom}}[\cdot]$, and $w_{\text{ctx}}[\cdot]$ that define the prior CTR for any opportunity:

$$z_0 = w_0 + w_{\text{ad}}[a] + w_{\text{dom}}[d] + w_{\text{ctx}}[c], \quad m_0 = \sigma(z_0) \in (0, 1),$$

where w_0 is the global intercept (baseline), and σ is the logistic function.

Our objective is to update these weights *gradually and safely* from data, using only **single-feature buckets** (not triples), so that priors re-calibrate over time without introducing instability in auctions.

3.1 Data Aggregation & Decay

For each family (ad category, domain type, context), we maintain decayed counts per bucket:

$$(I_k, C_k) \quad \text{for } k \in \mathcal{K}_{\text{family}},$$

updated once per day (or per hour) with exponential decay to adapt to distribution drift:

$$I_k^{(t)} \leftarrow \gamma I_k^{(t-1)} + I_{k,\text{today}}, \quad C_k^{(t)} \leftarrow \gamma C_k^{(t-1)} + C_{k,\text{today}},$$

with decay factor $\gamma \in (0, 1)$ (e.g. $\gamma = 0.97$ per day $\Rightarrow \sim 1$ month half-life). This same decay is applied to all feature families for consistency.

3.2 Smoothed CTR per Single-Feature Bucket

Each bucket k (e.g. `ad_category=health`) is modeled with a Beta prior based on its current weight and the global intercept:

$$m_{0,k} = \sigma(w_0 + w_{\text{fam}}[k]), \quad \alpha_{0,k} = m_{0,k} s_{\text{fam}}, \quad \beta_{0,k} = (1 - m_{0,k}) s_{\text{fam}},$$

where s_{fam} is the prior strength (pseudo-impressions) for this family (e.g. $s_{\text{ad}} = 200$, $s_{\text{dom}} = 200$, $s_{\text{ctx}} = 150$).

Given decayed counts (I_k, C_k) , the posterior parameters are:

$$\alpha'_k = \alpha_{0,k} + C_k, \quad \beta'_k = \beta_{0,k} + (I_k - C_k).$$

The *smoothed CTR* and its variance are:

$$\tilde{m}_k = E[\text{CTR}_k] = \frac{\alpha'_k}{\alpha'_k + \beta'_k}, \quad \text{Var}(\text{CTR}_k) = \frac{\alpha'_k \beta'_k}{(\alpha'_k + \beta'_k)^2 (\alpha'_k + \beta'_k + 1)}.$$

A 90% CI can be obtained either from Beta quantiles ($\text{BetaPPF}(0.05, \alpha'_k, \beta'_k)$, $\text{BetaPPF}(0.95, \alpha'_k, \beta'_k)$), or approximated by

$$CI_{90\%} \approx \tilde{m}_k \pm 1.645 \cdot \sqrt{\text{Var}(\text{CTR}_k)}.$$

3.3 Target Logit for the Bucket

We convert the smoothed CTR to a target *logit* for that bucket:

$$z_k = \text{logit}(\tilde{m}_k), \quad \text{logit}(p) = \ln \frac{p}{1-p}.$$

The corresponding *target weight* (relative to baseline) is

$$\Delta_k^* = z_k - w_0.$$

3.4 Adaptive Learning Rate

We move the weight $w_{\text{fam}}[k]$ toward the target with an evidence-scaled learning rate:

$$\eta_k = \lambda_{\text{fam}} \cdot \frac{I_k}{I_k + s_{\text{fam}}^{(\eta)}},$$

where $\lambda_{\text{fam}} \in (0, 1)$ is a small step-size (e.g. 0.1–0.5) and $s_{\text{fam}}^{(\eta)}$ is a level strength for the LR schedule (often set equal to s_{fam} for simplicity). Thus small buckets ($I_k \ll s_{\text{fam}}^{(\eta)}$) update slowly, while large buckets ($I_k \gg s_{\text{fam}}^{(\eta)}$) approach $\eta_k \approx \lambda_{\text{fam}}$.

3.5 Guardrails Before Each Update

We gate updates to avoid moving weights on noisy buckets:

CI/Variance Guard (per bucket). Compute the 90% CI length $CI_{\text{length},k}$ and compare with a threshold:

$$CI_{\text{length},k} = \text{high}_{90,k} - \text{low}_{90,k}, \quad CI_{\text{length},k} \leq CI_{\text{length},\max}^{\text{update}}.$$

Equivalently, using variance:

$$\text{Var}_{\max} = \left(\frac{CI_{\text{length},\max}^{\text{update}}}{2 \cdot 1.645} \right)^2, \quad \text{update allowed if } \text{Var}(\text{CTR}_k) \leq \text{Var}_{\max}.$$

Typical $CI_{\text{length},\max}^{\text{update}} = 0.02$ (2 percentage points).

Minimum Volume Guard (per bucket). Require $I_k \geq I_{\min}^{\text{update}}$ (e.g. $I_{\min}^{\text{update}} \in [500, 2000]$ depending on family and decay).

Only if *both* guards pass do we compute and apply a weight update.

3.6 Proposed Update, Clipping, and Ridge

Let $w_k \equiv w_{\text{fam}}[k]$ and $\Delta_k^* = z_k - w_0$. We compute the *proposed* change in weight:

$$\Delta w_k^{\text{prop}} = \eta_k \cdot (\Delta_k^* - w_k).$$

Clipping. To prevent shocks, clip the update:

$$\Delta w_k \leftarrow \text{clip}(\Delta w_k^{\text{prop}}, -\Delta_{\max}, +\Delta_{\max}), \quad \Delta_{\max} \approx 0.2 \text{ (log-odds)}.$$

Apply.

$$w_k \leftarrow w_k + \Delta w_k.$$

Ridge shrinkage (weight decay). A light L2-style pull to zero to avoid long-term drift:

$$w_k \leftarrow (1 - \rho) w_k, \quad \rho \in [0.01, 0.05].$$

3.7 Centering (Per Family)

After all buckets in a family are updated, re-center the family so its mean is zero:

$$\bar{w}_{\text{fam}} = \frac{1}{|\mathcal{K}_{\text{fam}}|} \sum_{k \in \mathcal{K}_{\text{fam}}} w_{\text{fam}}[k], \quad w_{\text{fam}}[k] \leftarrow w_{\text{fam}}[k] - \bar{w}_{\text{fam}}.$$

Centering preserves the interpretability of w_0 as the *true global baseline* and ensures each family's weights express relative lift/drop against its average.

3.8 Scheduling, Hyperparameters, and Storage

Cadence. Run the update job daily (or hourly for faster adaptation), after aggregations and decay are applied.

Defaults (typical starting values).

$\gamma = 0.97$ (daily), $s_{\text{ad}} = 200$, $s_{\text{dom}} = 200$, $s_{\text{ctx}} = 150$, $\lambda_{\text{fam}} = 0.3$, $s_{\text{fam}}^{(\eta)} = s_{\text{fam}}$,

$CI_{\text{length,max}}^{\text{update}} = 0.02$, $I_{\text{min}}^{\text{update}} \in [500, 2000]$, $\Delta_{\text{max}} = 0.2$, $\rho = 0.02$.

(These can be tuned per family as traffic grows.)

3.9 End-to-End Update Algorithm (Per Family)

1. Apply decay to (I_k, C_k) for all buckets k ; accumulate today's counts.
2. For each k :
 - (a) Compute $m_{0,k} = \sigma(w_0 + w_k)$; set $\alpha_{0,k} = m_{0,k}s_{\text{fam}}$, $\beta_{0,k} = (1 - m_{0,k})s_{\text{fam}}$.
 - (b) Posterior: $\alpha'_k = \alpha_{0,k} + C_k$, $\beta'_k = \beta_{0,k} + I_k - C_k$.
 - (c) Smoothed CTR: $\tilde{m}_k = \alpha'_k / (\alpha'_k + \beta'_k)$; variance via Beta formula.
 - (d) If $\text{Var}(\text{CTR}_k) > \text{Var}_{\text{max}}$ or $I_k < I_{\text{min}}^{\text{update}}$: *skip*.
 - (e) $z_k = (\tilde{m}_k)$; target $\Delta_k^* = z_k - w_0$.
 - (f) $\eta_k = \lambda_{\text{fam}} \cdot I_k / (I_k + s_{\text{fam}}^{(\eta)})$.
 - (g) $\Delta w_k^{\text{prop}} = \eta_k(\Delta_k^* - w_k)$; clip to $\pm\Delta_{\text{max}}$; apply.
 - (h) Ridge: $w_k \leftarrow (1 - \rho)w_k$.
3. Centering: subtract the family mean from all w_k .
4. Persist snapshots and weights.

3.10 Edge Cases and Safety

- **New buckets** (no history): start with $w_k = 0$, counts $(0, 0)$; guards will prevent updates until CI is narrow enough or volume builds.
- **Numerical safety**: clamp probabilities to $[10^{-8}, 1 - 10^{-8}]$ before $(\cdot)$.
- **Optional hard bounds**: cap weights to $w_k \in [-2, +2]$ (CTR multipliers remain reasonable).

3.11 Worked Example

Assume global baseline CTR = 1.00% so $w_0 = (0.01) \approx -4.595$. Initialize family **ad category** weights to zero for three buckets: **health**, **finance**, **travel**. Let $s_{\text{ad}} = 200$, $CI_{\text{length,max}}^{\text{update}} = 0.02$, $\lambda_{\text{ad}} = 0.3$, $\rho = 0.02$, $\Delta_{\text{max}} = 0.2$. Suppose decayed counts for the day are:

Bucket	I	C
health	8000	120
finance	5000	45
travel	2000	18

Health. $m_{0,\text{health}} = \sigma(-4.595 + 0) = 0.01$, $\alpha_0 = 2, \beta_0 = 198$. Posterior: $\alpha' = 122, \beta' = 8078$. Smoothed CTR: $\tilde{m} = 122/8200 \approx 1.488\%$. $z = (0.01488) \approx -4.193$. Target rel. weight: $\Delta^* = z - w_0 \approx 0.402$. Learning rate: $\eta = 0.3 \cdot \frac{8000}{8000+200} \approx 0.293$. Proposed update: $\Delta w^{\text{prop}} = 0.293 \cdot (0.402 - 0) \approx 0.118$. Clip (not needed): $\Delta_{\max} = 0.2$. Apply and ridge: $w \leftarrow 0 + 0.118$ then $w \leftarrow (1 - 0.02) \cdot 0.118 \approx 0.115$.

Finance. $m_0 = 0.01, \alpha_0 = 2, \beta_0 = 198$. Posterior: $\alpha' = 47, \beta' = 5153$. $\tilde{m} \approx 0.904\%$, $z \approx -4.697$. $\Delta^* \approx -0.102$. $\eta \approx 0.288$. $\Delta w^{\text{prop}} \approx 0.288 \cdot (-0.102 - 0) \approx -0.029$. Ridge: $w \approx -0.029 \cdot 0.98 \approx -0.029$.

Travel. $m_0 = 0.01, \alpha_0 = 2, \beta_0 = 198$. Posterior: $\alpha' = 20, \beta' = 2180$. $\tilde{m} \approx 0.909\%$, $z \approx -4.691$. $\Delta^* \approx -0.096$. $\eta \approx 0.273$. $\Delta w^{\text{prop}} \approx -0.026$. Ridge: $w \approx -0.026 \cdot 0.98 \approx -0.026$.

Centering. Weights after ridge (pre-centering): health = +0.115, finance = -0.029, travel = -0.026. Family mean $\bar{w} = (+0.115 - 0.029 - 0.026)/3 \approx +0.020$. Subtract mean:

$$w_{\text{health}} = +0.115 - 0.020 \approx +0.095, \quad w_{\text{finance}} = -0.029 - 0.020 \approx -0.049, \quad w_{\text{travel}} = -0.026 - 0.020 \approx -0.046.$$

Interpretation: with a 1% baseline, the prior CTR for the health ad category lifts slightly (about +0.05 pp around baseline), while finance/travel drop slightly; all updates were allowed because their CI widths were well below 2 pp and volumes above $I_{\min}^{\text{update}}$.

This completes one full, guarded update cycle. Repeating this daily keeps priors aligned with observed performance while maintaining stability, interpretability, and auction safety.

4 Design Rationale for the Weight Update Pipeline

This section explains *why* each element of the update pipeline is required: evidence-scaled learning rate, CI/variance guards, per-step clipping, ridge regularization, and family centering. Together they balance **stability** and **adaptation**, guaranteeing that priors evolve with data without destabilizing auctions or miscalibrating dashboards.

4.1 Evidence-Scaled Learning Rate

We update a bucket weight w_k toward its target logit $\Delta_k^* = z_k - w_0$ via

$$w_k^{\text{new}} = (1 - \eta_k) w_k + \eta_k \Delta_k^*, \quad \eta_k = \lambda_{\text{fam}} \cdot \frac{I_k}{I_k + s_{\text{fam}}^{(\eta)}}.$$

Why this form?

- **Evidence-proportional trust.** As impressions I_k grow, η_k rises monotonically to an upper bound λ_{fam} , letting high-traffic buckets adapt faster while sparse buckets move slowly.
- **Bayesian analogy.** Posterior means combine prior and empirical estimates with weights proportional to prior strength vs. data size; here η_k plays the analogous role at the logit level.
- **Contraction.** With $\eta_k \in (0, 1)$, the map $w_k \mapsto w_k^{\text{new}}$ is a contraction toward Δ_k^* (no overshoot by design).

If omitted: weights overreact to day-to-day noise (especially at small I_k), causing volatile priors and unstable eCPM in auctions.

4.2 CI/Variance Guard (Only Update When the Estimate is Tight Enough)

Before any update we require both:

$$I_k \geq I_{\min}^{\text{update}} \quad \text{and} \quad CI_{\text{length},k} \leq CI_{\text{length},\max}^{\text{update}}.$$

Equivalently via variance,

$$\text{Var}(\text{CTR}_k) \leq \text{Var}_{\max}, \quad \text{Var}_{\max} = \left(\frac{CI_{\text{length},\max}^{\text{update}}}{2 \cdot 1.645} \right)^2.$$

Why this guard?

- **Noise gating.** Prevents learning from buckets whose posterior CTR is still too uncertain (wide CI), regardless of I_k .
- **Consistency across families.** A universal CI threshold (in percentage points) gives a comparable quality bar across ad/domain/context families with different scales.

If omitted: you may “learn” from random spikes (e.g., 1 click on 20 impressions), pushing weights incorrectly and whipsawing priors.

4.3 Per-Update Clipping (Shock Absorber)

After computing the proposed change

$$\Delta w_k^{\text{prop}} = \eta_k (\Delta_k^* - w_k),$$

we enforce

$$\Delta w_k \leftarrow \text{clip}(\Delta w_k^{\text{prop}}, -\Delta_{\max}, +\Delta_{\max}) \quad \text{with} \quad \Delta_{\max} \approx 0.2 \text{ (log-odds)}.$$

Why clipping?

- **Outlier robustness.** Even with guards, occasional outliers (traffic anomalies, partial outages) can imply large Δw ; clipping caps per-cycle movement.
- **Lipschitz control.** Ensures per-iteration change is bounded, improving numerical stability and preventing oscillation.

If omitted: rare events can cause large weight jumps that take many cycles to undo, degrading auction ranking and reported CTRs.

4.4 Ridge Regularization (Weight Decay)

After applying the (clipped) update:

$$w_k \leftarrow (1 - \rho) w_k, \quad \rho \in [0.01, 0.05].$$

Why ridge?

- **Prevent long-term drift.** In non-stationary environments, small systematic biases accumulate; ridge gently pulls weights back toward zero unless consistently supported by new data.
- **L2 prior view.** This is equivalent to a zero-mean Gaussian prior on w_k (L2 penalty), controlling variance of weights and reducing overfit.
- **Forgetting without resets.** The effective half-life is $\approx \ln 2 / \rho$ update cycles; pick ρ to match how fast you want stale signals to fade.

If omitted: weights may “stick” at inflated magnitudes due to historical conditions no longer present, biasing priors and auctions.

4.5 Family Centering (Identifiability and Interpretability)

After updating all buckets in a family, subtract the mean:

$$\bar{w}_{\text{fam}} = \frac{1}{|\mathcal{K}_{\text{fam}}|} \sum_k w_{\text{fam}}[k], \quad w_{\text{fam}}[k] \leftarrow w_{\text{fam}}[k] - \bar{w}_{\text{fam}}.$$

Why centering?

- **Keep w_0 as the true baseline.** With centering, the family contributes zero mean shift, so w_0 remains the logit of global CTR; weights become *relative* lifts/drops vs. family average.
- **Avoid double counting.** Without centering, entire families can drift up/down together and w_0 will also move, effectively counting the same global trend twice.
- **Numerical conditioning.** Zero-mean features reduce collinearity and improve stability of subsequent learning.

If omitted: intercept interpretability is lost, global shifts leak into family weights, and comparisons across categories become misleading.

4.6 Operational Ordering and Interplay

Order matters:

1. Compute posterior CTR, variance, CI.
2. Check update guards ($I_{\min}^{\text{update}}$ and CI/variance).
3. Compute η_k and Δw_k^{prop} .
4. Apply clipping to Δw_k ; update w_k .
5. Apply ridge decay $w_k \leftarrow (1 - \rho)w_k$.
6. Center the family (mean-zero).

This sequencing ensures: (i) we never update on low-quality estimates, (ii) step sizes are bounded, (iii) drift is continuously counteracted, and (iv) the intercept w_0 stays interpretable.

4.7 What Breaks If You Remove a Piece?

- **No learning rate:** violent day-to-day swings; sparse buckets dominate with noise.
- **No CI guard:** you “learn” from random spikes; instability in auctions and dashboards.
- **No clipping:** rare outliers create large, persistent miscalibration.
- **No ridge:** weights fossilize at outdated values; slow drift corrupts priors.
- **No centering:** w_0 stops being the baseline; families drift together; relative interpretation is lost.

5 Baseline Parameter w_0

The global intercept w_0 is the anchor of the entire CTR modeling system. It represents the **global baseline CTR** of the platform in log-odds space and ensures that all feature weights are interpreted as *relative deviations* against this baseline. If w_0 drifts or loses interpretability, every other component (weights, priors, auctions, dashboards) becomes miscalibrated. This section defines what w_0 is, how to initialize it, how to update it, and why it is essential.

5.1 Definition and Interpretation

CTR is expressed as a probability $p \in (0, 1)$. We map this into log-odds space via the *logit*:

$$\text{logit}(p) = \ln \frac{p}{1-p}.$$

The baseline is then:

$$w_0 = \text{logit}(m_{\text{global}}),$$

where m_{global} is the expected global CTR.

Interpretation:

- $\sigma(w_0) = \frac{1}{1+e^{-w_0}} = \text{global average CTR}$.
- Each feature weight $w_{\text{fam}}[k]$ is a deviation in log-odds relative to this baseline.
- Example: if $w_0 = -4.595$ (corresponding to CTR = 1%), and $w_{\text{ad}}[\text{health}] = +0.06$, then the CTR for health ads is:

$$\sigma(-4.595 + 0.06) \approx 1.05\%.$$

5.2 Initialization (Cold Start)

At platform start, we define a prior for global CTR using a Beta distribution:

$$CTR_{\text{global}} \sim \text{Beta}(\alpha_{0,g}, \beta_{0,g}),$$

with parameters set by prior mean $m_{0,g}$ (e.g. 1%) and prior strength s_g (e.g. 500 pseudo-impressions):

$$\alpha_{0,g} = m_{0,g} \cdot s_g, \quad \beta_{0,g} = (1 - m_{0,g}) \cdot s_g.$$

The prior mean is:

$$m_{\text{global}}^{(0)} = \frac{\alpha_{0,g}}{\alpha_{0,g} + \beta_{0,g}},$$

and the initial intercept is:

$$w_0^{(0)} = \left(m_{\text{global}}^{(0)} \right).$$

5.3 Posterior Update with Data

After observing I impressions and C clicks:

$$\alpha' = \alpha_{0,g} + C, \quad \beta' = \beta_{0,g} + (I - C).$$

The posterior mean CTR is:

$$m_{\text{global}} = \frac{\alpha'}{\alpha' + \beta'},$$

and the updated baseline is:

$$w_0 = (m_{\text{global}}).$$

Because the global CTR is estimated from very large I , variance is low:

$$\text{Var}(CTR_{\text{global}}) = \frac{\alpha' \beta'}{(\alpha' + \beta')^2 (\alpha' + \beta' + 1)}.$$

5.4 Update Cadence and Stability

Two approaches can be used:

- **Direct recomputation:** Simply recompute w_0 daily from the global posterior mean.
- **Exponential moving average (EMA):**

$$w_0^{\text{new}} = (1 - \eta_g) w_0^{\text{old}} + \eta_g (m_{\text{global}}),$$

with $\eta_g \in [0.01, 0.05]$. This prevents jitter but reacts to long-term shifts.

5.5 Guardrails and CI

To avoid miscalibration:

- Compute 90% credible intervals for m_{global} :

$$CI_{90\%} = [\text{BetaPPF}(0.05, \alpha', \beta'), \text{BetaPPF}(0.95, \alpha', \beta')].$$

- Only adjust w_0 if CI width is below a small threshold (e.g. 0.5 percentage points). This ensures the baseline moves only when the global CTR is estimated with high certainty.

5.6 Interaction with Feature Weights

Centering ensures that:

$$\sum_k w_{\text{fam}}[k] = 0 \quad \forall \text{ families.}$$

This guarantees that w_0 alone reflects the true global CTR. Without centering, w_0 would absorb family drift, losing its meaning.

5.7 Role in Auctions

CTR predictions are always computed relative to w_0 :

$$z = w_0 + w_{\text{ad}}[a] + w_{\text{dom}}[d] + w_{\text{ctx}}[c].$$

If w_0 is wrong, every eCPM calculation is biased across the board, leading to mispriced auctions and unfair payouts. A stable and correct w_0 guarantees that effective CPMs remain consistent across pricing models (CPC vs CPM).

5.8 Role in Dashboards

Dashboards display:

- The baseline CTR: $\sigma(w_0)$, as the global reference.
- Relative lifts/drops for feature families (e.g. “health ads perform +0.05 pp better than average”).

This interpretability depends entirely on w_0 anchoring the true global average.

5.9 Role in Model Training

When training ML models for CTR prediction, the baseline w_0 provides:

- A global prior, ensuring even zero-shot buckets (new ad category, new publisher) start from the correct baseline.
- Weight stability, since centered feature weights are relative to w_0 .
- Guardrails: training excludes buckets where CI is too wide, ensuring w_0 does not drift based on noisy inputs.

5.10 Summary

- w_0 is the logit of the global CTR and the **anchor of the entire system**.
- Initialize with Beta prior parameters from guessed mean and strength.
- Update with posterior mean (or EMA) from global impressions and clicks.
- Apply CI guardrails to avoid jitter.
- Center families so w_0 remains interpretable.
- Without a correct and stable w_0 , feature weights drift, CTR estimates lose calibration, auctions misprice, dashboards misreport, and ML models degrade.

Background Reminder: Bernoulli, Binomial, and Conjugacy

Bernoulli Distribution. The Bernoulli distribution models a single binary trial (success/failure).

$$X \sim \text{Bernoulli}(p), \quad P(X = 1) = p, \quad P(X = 0) = 1 - p.$$

In our context: one impression $\rightarrow$ clicked (1) or not clicked (0).

Binomial Distribution. The Binomial distribution is the sum of n independent Bernoulli trials.

$$C \sim \text{Binomial}(n, p), \quad P(C = c) = \binom{n}{c} p^c (1-p)^{n-c}.$$

In our context: after n impressions, the number of clicks C follows a Binomial distribution with parameter $p = \text{true CTR}$.

Conjugacy. In Bayesian statistics, a prior $p(\theta)$ and likelihood $p(\mathcal{D} \mid \theta)$ are said to be **conjugate** if the posterior $p(\theta \mid \mathcal{D})$ belongs to the same family as the prior.

- For a Bernoulli/Binomial likelihood, the **conjugate prior** is $\text{Beta}(\alpha, \beta)$.
- This means: if $p \sim \text{Beta}(\alpha, \beta)$ and data $C \sim \text{Binomial}(n, p)$ are observed, then

$$p \mid C \sim \text{Beta}(\alpha + C, \beta + n - C).$$

- Thus, updating beliefs about CTR is a simple matter of incrementing α by observed clicks and β by observed non-clicks.

Why this matters for CTR. CTR is exactly a Bernoulli/binomial process:
 - Each impression = Bernoulli trial. - Aggregated impressions = Binomial. - Beta is conjugate to Binomial, so we can keep closed-form updates.

This is why the Beta–Binomial framework is the mathematically natural and computationally efficient way to model CTR.

6 Why Beta Distributions for CTR Modeling?

CTR modeling in Adstract uses the Beta distribution as the foundation for all priors and posteriors. This choice is not arbitrary: the Beta distribution is mathematically natural for Bernoulli/binomial processes, interpretable, and computationally efficient. This section explains why the Beta is used, and what advantages it provides over naive frequency estimates.

6.1 CTR as a Bernoulli Process

CTR is defined as

$$CTR = \frac{\text{clicks}}{\text{impressions}}.$$

Each impression is a Bernoulli trial (click = 1, no click = 0). The sequence of impressions thus follows a Binomial distribution:

$$C \sim \text{Binomial}(I, p),$$

where I is impressions, C is clicks, and p is the true underlying CTR.

6.2 Conjugacy of Beta and Binomial

The Beta distribution is the *conjugate prior* for the Binomial likelihood. If $p \sim \text{Beta}(\alpha, \beta)$ and we observe C clicks from I impressions, the posterior is:

$$p \mid C, I \sim \text{Beta}(\alpha + C, \beta + I - C).$$

This closed-form update is extremely efficient and does not require approximate inference or heavy computation. It allows us to update CTR beliefs incrementally in real-time.

6.3 Interpretability as Pseudo-Counts

The parameters (α, β) can be read as *pseudo-clicks* and *pseudo-non-clicks*:

$$\alpha = \text{prior clicks}, \quad \beta = \text{prior non-clicks}.$$

Thus:

- $\alpha + \beta$ = strength of belief (pseudo-impressions).
- $\frac{\alpha}{\alpha + \beta}$ = prior mean CTR.

This makes priors easy to set (e.g. “assume CTR = 1% with 500 pseudo-impressions of strength”) and easy to explain to non-technical stakeholders.

6.4 Variance and Confidence Intervals

The Beta distribution provides closed-form variance:

$$\text{Var}(p) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}.$$

This allows us to quantify uncertainty directly, and to derive credible intervals for guardrails:

$$CI_{90\%} = [\text{BetaPPF}(0.05, \alpha, \beta), \text{BetaPPF}(0.95, \alpha, \beta)].$$

CTR estimates are therefore not only point predictions, but also come with an interpretable measure of confidence.

6.5 Smooth Transition from Prior to Empirical

A naive CTR estimate C/I is unstable at small I . The Beta posterior mean:

$$\widehat{CTR} = \frac{\alpha + C}{\alpha + \beta + I}$$

smoothly interpolates between the prior and the empirical fraction. When I is small, the prior dominates. When I is large, the empirical fraction dominates. This avoids wild swings at low volume.

6.6 Practical Advantages

- **Closed-form updates:** CTR estimates update in $O(1)$ time per observation.
- **Statistical meaning:** α, β have intuitive interpretations as pseudo-counts.
- **Guardrails:** Natural variance and CI allow us to apply trust rules and avoid learning from noise.
- **Extensibility:** The Beta generalizes easily into hierarchical Bayesian setups, enabling shrinkage across ad categories, domains, and contexts.
- **Compatibility with auctions:** Expected CTRs can be plugged directly into eCPM formulas for CPC/CPM unification.

6.7 Summary

The Beta distribution is the **natural statistical model for CTR**:

- CTR is fundamentally binomial; Beta is its conjugate.
- Priors and updates are easy to compute and interpret.
- Variance and CI fall out naturally, enabling robust guardrails.
- It provides a smooth and reliable foundation for both cold start and mature traffic.

Without Beta modeling, CTR estimates would either be unstable (naive counts) or computationally expensive (non-conjugate Bayesian methods).

7 Future Improvements: Hierarchical Shrinkage and ML Models

While the current framework (Beta priors with Bayesian smoothing and guarded weight updates) provides a robust and interpretable MVP for CTR modeling, future improvements can substantially increase accuracy, adaptability, and contextual awareness. Two directions are especially promising: *hierarchical shrinkage* and *machine learning models*.

7.1 Hierarchical Shrinkage

At present, priors for each bucket (ad category, domain type, context) are updated independently. This leads to sparse-data issues: low-traffic buckets update very slowly, while high-traffic buckets adapt quickly. A natural extension is to use **hierarchical Bayesian shrinkage**, where estimates for fine-grained buckets “borrow strength” from broader aggregates.

Idea.

- The CTR for an ad in category a , domain d , context c is modeled as:

$$p_{a,d,c} \sim \text{Beta}(\alpha_{a,d,c}, \beta_{a,d,c}),$$

but $\alpha_{a,d,c}, \beta_{a,d,c}$ themselves are informed by higher-level posteriors.

- For example:

$$\alpha_{a,d,c} = \rho_1 \alpha_a + \rho_2 \alpha_d + \rho_3 \alpha_c,$$

with similar structure for β . Here ρ are shrinkage weights.

- Thus, low-volume cells shrink toward their family mean (ad, domain, context), and those shrink further toward the global baseline.

Benefits.

- Improves stability in rare buckets (e.g., a niche ad category in a small context).
- Provides more accurate CTR estimates early, before sufficient local impressions accumulate.
- Ensures consistency across levels: no bucket can deviate wildly without strong evidence.

Implementation Options.

- **Recursive shrinkage:** compute posterior means at global, family, and bucket levels, and recursively blend them.
- **Weighted blending:** combine priors from multiple levels (ad, domain, context) with weights proportional to their precision (inverse variance).

This hierarchical extension maintains closed-form updates while addressing sparse-data challenges.

7.2 Machine Learning Models for CTR Prediction

Long-term, we move beyond smoothed ratios to proper predictive models. ML-based CTR estimation can incorporate richer context and nonlinear interactions that the current additive logit model cannot capture.

Motivation.

- CTR is influenced not only by static features (ad category, domain, context), but also by dynamic signals (time of day, recency, conversation semantics).
- Simple Bayesian smoothing does not capture interactions or feature embeddings.

Approach.

- Treat CTR prediction as a supervised learning task:

$$y \in \{0, 1\} \quad \text{click vs no-click.}$$

- Train models such as:
 - Logistic regression with embeddings (fast, interpretable).
 - Gradient Boosted Trees (handles categorical splits well).
 - Deep models (contextual embeddings of prompts, user signals).
- Input features:
 - Ad-level: category, historical CTR, creative features.
 - Domain-level: publisher type (vertical/horizontal), historical CTR.
 - Context-level: conversation category, semantic embeddings of prompt.
 - Temporal: day of week, hour of day.

Guardrails. The same guard conditions apply for training data: only buckets where

$$I \geq I_{\min}, \quad CI_{\text{length}} \leq CI_{\text{length}, \max}$$

are included. Samples can be weighted by inverse variance ($1/\text{Var}$), ensuring stable buckets dominate training.

Benefits.

- Captures nonlinear interactions between features.
- Generalizes to unseen contexts (e.g. new publishers, new ad categories) via embeddings.
- Increases auction efficiency: CPC vs CPM unification is more precise when CTR predictions are better calibrated.

7.3 Summary of Future Work

- **Hierarchical shrinkage:** ensures low-traffic buckets get robust estimates by borrowing strength from broader levels.
- **ML models:** allow richer contextualization, embeddings, and predictive power beyond simple ratios.

Both enhancements build on the Bayesian framework already in place: the MVP provides safe, interpretable estimates now, while future work scales accuracy and adaptability as data grows.

8 Conclusion

CTR modeling is the cornerstone of any advertising system, and in the Adstrack platform it is even more critical given the contextual nature of LLM-based ad placements. Unlike static display advertising, chatbot integrations operate at smaller volumes, higher variance, and with strong dependencies on publisher domain and conversation context. These factors make naive click/impression ratios insufficient and highlight the need for a principled statistical foundation.

In this paper we presented the full CTR modeling pipeline for Adstrack:

- A **Bayesian Beta-Binomial framework** that provides well-calibrated priors at cold start, smooth posteriors with low-volume data, and interpretable pseudo-count parameters.
- **Variance- and CI-based guardrails** to determine when estimates can be trusted, ensuring that auctions, dashboards, and model training are shielded from noise.
- A structured procedure for **weight updates**, including evidence-scaled learning rates, clipping, ridge regularization, and centering. This stabilizes feature priors while preserving interpretability.
- The global intercept w_0 **as a baseline anchor**, maintaining consistency across families, ensuring fair eCPM calculations, and preserving dashboard clarity.

This design delivers a robust Minimum Viable Product (MVP) that is:

- Statistically principled yet computationally efficient.
- Interpretable for stakeholders across engineering, product, and business.
- Safe for auctions and reporting from the very first impression.

Looking forward, the framework is readily extensible. **Hierarchical shrinkage** can further improve low-volume estimates by borrowing strength across levels, while **machine learning models** can incorporate nonlinear interactions and semantic embeddings to increase predictive power. Both enhancements naturally build upon the Bayesian foundation presented here.

In summary, Adstrack’s CTR modeling pipeline brings the rigor of established ad-tech practices into the novel setting of contextual LLM responses. It ensures that even in a high-variance, low-volume environment, CTR estimates remain stable, interpretable, and actionable. This is the statistical core that enables fair pricing, transparent dashboards, and trustworthy optimization—forming the backbone of a scalable advertising platform.